Incident Triage Agent

Architecture, Working Flow & Prediction Engine

A detailed technical walkthrough

1. Project Overview

The Incident Triage Agent is an AI-powered system that automatically diagnoses production alerts, identifies root causes, and pages the correct on-call engineer — all without human intervention. It also **predicts incidents before they occur** by continuously monitoring error-rate trends across all services.

The stack: **React + TypeScript** frontend, **FastAPI** backend, **LangGraph** state machine, **Azure OpenAI GPT** for reasoning, **FAISS** vector store for runbook retrieval, and five mock microservices (Jira, Splunk, ServiceNow, Slack, On-Call).

2. System Architecture

Component	Port	Role
React UI	5173	Live dashboard — alert feed, agent flow panel, triage report
FastAPI Backend	8000	REST API + WebSocket hub + LangGraph pipeline orchestrator
Mock Jira	8001	Historical incident tickets (past + currently open)
Mock Splunk/ELK	8002	Application logs + error-rate trend data for predictions
Mock ServiceNow	8003	ITSM tickets — search, get, create
Mock Slack	8004	Receives and stores formatted incident reports
Mock On-Call	8005	Shift-based engineer routing + page events

Communication topology:

```
React UI (5173)
    ■  WebSocket (ws://)  +  REST fetch (via Vite proxy)
```

```
    ▼
FastAPI Backend (8000)
    ■  MCP Gateway  (tool calls — never direct HTTP from agents)
    ■■■■ Jira (8001)       search_past_incidents
    ■■■■ Splunk (8002)     search_logs  /  get_log_trends
    ■■■■ ServiceNow (8003) search_servicenow_incidents / create_incident_ticket
    ■■■■ Slack (8004)      post_slack_report
    ■■■■ On-Call (8005)    get_oncall_engineer / page_oncall_engineer
```

3. Reactive Triage Flow (Alert → Resolution)

Step 1 — Alert Ingestion

An external monitoring system (or the test script) sends a JSON payload to **POST /alert**. The payload contains the service name, alert type, severity, affected endpoints, and current metrics.

```
python scripts/push_alert.py --scenario db_timeout
  ■
  ■■■ POST http://localhost:8000/alert
        {
          "service":   "payment-service",
          "alert_type": "DB_CONNECTION_TIMEOUT",
          "severity":  "P1",
          "metrics":   {"error_rate": "23%", "latency_p99": "8200ms"}
        }
```

The backend immediately:

- Creates an incident record in memory with status **ingested**.
- Broadcasts a **new_alert** WebSocket event — the UI card appears instantly.
- Returns HTTP 202 Accepted (non-blocking).
- Spawns the triage pipeline as a background asyncio task.

Step 2 — LangGraph State Machine

The triage pipeline is a **directed acyclic graph** built with LangGraph. Each node is an async function that reads from and writes to a shared **IncidentState** dictionary. The graph compiles to a runnable that streams intermediate state updates — every node completion fires a WebSocket broadcast so the UI agent-flow panel updates in real time.

```
START
  ■
  ▼
ingest_node          ← resets flags, records start time
  ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ▼                         ▼                        ▼
log_analyzer_node    past_ticket_node         runbook_node
(Splunk logs)        (Jira + ServiceNow)      (FAISS / RAG)
  ■                         ■                        ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                           ■   fan-in (all 3 must finish)
                           ▼
```

```
                root_cause_node     ← Azure OpenAI GPT
                      ■
                      ▼
                report_node         ← on-call lookup, page, Slack post
                      ■
                    END
```

Step 3 — Parallel Evidence Gathering

Three agents run **simultaneously** after ingest_node completes:

Agent	Tool Called	Data Fetched
LogAnalyzerAgent	search_logs → Splunk (8002)	Error + FATAL logs for the service in the last 30 min. Includes scenario-specific anchor logs injected by the mock.
PastTicketAgent	search_past_incidents → Jira (8001) search_servicenow_incidents → ServiceNow (8003)	Historical resolved incidents matching the service + alert type. Also returns currently OPEN / IN-PROGRESS tickets.
RunbookAgent	RAGRetriever.search() → local FAISS index	Top-5 semantically similar runbook chunks. Embedded at startup using Azure OpenAI Embeddings API.

Step 4 — Root Cause Synthesis (Azure OpenAI GPT)

Once all three parallel agents finish, **root_cause_node** calls Azure OpenAI with a structured prompt that bundles all gathered evidence:

```
=== ALERT DETAILS ===         ← original alert payload
=== LOG ANALYSIS FINDINGS === ← Splunk error patterns
=== SIMILAR PAST INCIDENTS === ← resolved Jira + ServiceNow tickets
=== CURRENTLY OPEN TICKETS === ← active in-progress tickets (!)
=== RUNBOOK GUIDANCE ===      ← RAG-retrieved runbook text
```

The LLM (temperature=0) responds with strict JSON:

- **hypotheses** — Top 3 root cause hypotheses ranked by confidence (High/Medium/Low), each with evidence quotes and remediation steps.
- **remediation_checklist** — 10 prioritised actions with owner and time estimate per step.
- **escalation** — Boolean required flag, priority level, team to escalate to, and reason.

Open tickets are critical inputs here — if Jira/ServiceNow already has an active investigation for the same service and error code, the LLM references those ticket IDs in its evidence and remediation steps.

Step 5 — Report Generation, On-Call Paging & Slack

report_node assembles the final report and performs three actions:

Action	How	Condition
On-call lookup	GET /api/oncall/current?service=<name> Mock On-Call (8005)	Always — resolves the team and current shift engineer (morning/aftern
Auto-page engineer	POST /api/oncall/page Mock On-Call (8005)	Only if: escalation.required=true AND severity=P1
Post to Slack	POST /api/slack/post Mock Slack (8004)	Always — full structured report sent to the incident channel

Every state change throughout steps 1–5 is broadcast over WebSocket so the React UI updates live: the agent-flow panel shows each node going from pending → running → completed, and the progress bar advances in real time.

Step 6 — Post-Mortem (human-triggered)

After the incident reaches **done** status, an engineer clicks Resolve in the UI, which calls **POST /incidents/{id}/resolve** with a resolution summary, confirmed root cause, and who resolved it. **PostMortemAgent** runs another Azure OpenAI call to generate a full post-mortem document, stored at **GET /post-mortems/{id}**.

4. Predictive Monitoring (Before the alert fires)

The predictive path runs completely independently of the reactive triage pipeline. It is a background asyncio loop that starts at application startup and polls Splunk trend data for all monitored services on a configurable interval.

4.1 How it starts

```
# backend/main.py — FastAPI lifespan startup
asyncio.create_task(run_predictive_monitor())

async def run_predictive_monitor():
    await asyncio.sleep(15)   # wait for mocks to start
    while True:
        agent = PredictiveMonitorAgent(mcp_gateway=MCPGateway())
        predictions = await agent.run()   # check all 5 services
        for pred in predictions:
            await manager.broadcast({"event": "prediction", ...})
        await asyncio.sleep(interval)     # configured in .env
```

4.2 Per-service trend check

For each of the 5 monitored services, the agent calls **get_log_trends** via the MCP Gateway, which hits **GET /api/logs/trends?service=<name>** on Mock Splunk (8002). The response contains:

Field	Meaning
current_error_rate_pct	Live error rate (%) right now
baseline_error_rate_pct	Normal/expected error rate for this service
trend	"increasing" | "stable-elevated" | "stable"
rate_of_change_per_min	How fast the error rate is growing (% per minute)
anomaly_score	0.0–1.0 composite anomaly score
predicted_threshold_breach_minutes	ETA until SLA threshold is breached
top_error_codes	e.g. ["CONN_TIMEOUT_5023", "DB_POOL_EXHAUSTED"]

4.3 Prediction decision logic

The agent fires a prediction if **ANY ONE** of three conditions is true:

```
ANOMALY_THRESHOLD        = 0.55   # anomaly_score > 0.55
ERROR_RATE_MULTIPLIER    = 2.5    # current > baseline × 2.5
RATE_OF_CHANGE_THRESHOLD = 0.25   # roc > 0.25 %/min AND trend = "increasing"

def _should_predict(trend):
    return (
        anomaly_score > 0.55
        OR current_error_rate > baseline * 2.5
        OR (trend == "increasing" AND rate_of_change > 0.25)
    )
```

4.4 Confidence and severity mapping

anomaly_score range	Confidence	Predicted Severity
> 0.8	High	P2
0.6–0.8	Medium	P3
< 0.6	Low	P3

4.5 Alert type inference from error codes

The top error codes from the trend response are mapped to known alert types:

```
CONN_TIMEOUT_5023   →  DB_CONNECTION_TIMEOUT
DB_POOL_EXHAUSTED   →  DB_CONNECTION_TIMEOUT
CACHE_MISS_HIGH     →  HIGH_ERROR_RATE
OOM_ERROR           →  MEMORY_LEAK
HEAP_HIGH           →  MEMORY_LEAK
NPE_CHECKOUT        →  DEPLOY_REGRESSION
DEPENDENCY_503      →  DEPENDENCY_FAILURE
SMS_TIMEOUT         →  DEPENDENCY_FAILURE
```

4.6 How the mock generates spikes (demo realism)

The Mock Splunk trends endpoint injects random spikes so predictions fire occasionally but not on every poll cycle. The spike logic:

```
# mock_splunk.py — /api/logs/trends
spike = random.choice([0, 0, 0, random.uniform(2.0, 6.0)])
# 3 out of 4 polls → no spike (stable)
# 1 out of 4 polls → spike of 2x–6x baseline injected

anomaly = (current - baseline) / baseline * 0.5 + random.uniform(0, 0.25)
# anomaly_score is proportional to how far current deviates from baseline
```

4.7 What happens in the UI when a prediction fires

The backend broadcasts a **prediction** WebSocket event. The React UI adds a new ■■ card to the alert feed (visually distinct from real incidents). Selecting it shows a prediction detail view — no agent pipeline runs, because no incident has occurred yet. If a real alert later fires for the same service, it appears as a separate incident card and the full triage pipeline runs.

5. RAG Pipeline (Runbook Retrieval)

Retrieval-Augmented Generation provides the LLM with relevant runbook knowledge at triage time, grounding its reasoning in actual operational procedures.

Build phase (runs once at startup if index missing):

```
backend/rag/pipeline.py
  1. Read runbook text files from backend/data/
  2. Split into overlapping chunks (~500 tokens each)
  3. Call Azure OpenAI Embeddings API → each chunk → float32 vector
  4. Store vectors in FAISS flat index
  5. Save index.faiss + metadata.pkl to backend/rag/faiss_index/
```

Query phase (runs inside runbook_node per incident):

```
RAGRetriever.search(query="DB_CONNECTION_TIMEOUT payment-service")
  1. Embed the query string via Azure OpenAI Embeddings
  2. FAISS.search(query_vector, top_k=5)  ← cosine-like L2 nearest-neighbour
  3. Return top-5 chunks ranked by similarity score
  4. Chunks are concatenated and passed as "RUNBOOK GUIDANCE" in the LLM prompt
```

6. MCP Gateway (The Tool Bus)

Every external call — Splunk, Jira, ServiceNow, Slack, On-Call — goes through **MCPGateway.call_tool(name, params)**. Agents never make HTTP calls directly. This is the **Model Context Protocol** pattern: a registry of named, typed tools with JSON schemas, making it trivial to swap mock services for real ones.

Tool Name	Routes To	Used By
search_logs	Splunk /api/logs/search	LogAnalyzerAgent
get_log_trends	Splunk /api/logs/trends	PredictiveMonitorAgent
search_past_incidents	Jira /api/incidents/search	PastTicketAgent
search_servicenow_incidents	ServiceNow /api/incidents/search	PastTicketAgent
get_incident_details	ServiceNow /api/incidents/{id}	PastTicketAgent
create_incident_ticket	ServiceNow POST /api/incidents	available for future use
post_slack_report	Slack /api/slack/post	report_node
get_oncall_engineer	On-Call /api/oncall/current	report_node
page_oncall_engineer	On-Call /api/oncall/page	report_node (P1 auto-page)

7. WebSocket Real-Time Event Flow

The React UI maintains a persistent WebSocket connection to **ws://localhost:8000/ws** with automatic 3-second reconnection. The backend broadcasts the following event types:

Event	When fired	UI reaction
connected	Client connects	Fetch full incident list
new_alert	POST /alert received	New incident card appears, selected

incident_update	Each pipeline node completes	Progress bar, agent steps update
prediction	PredictiveMonitor fires	■■ prediction card appears
post_mortem_complete	PostMortemAgent finishes	Incident status → resolved

8. End-to-End Summary

Path	In one sentence
Reactive triage	Alert in → 3 agents gather evidence in parallel (Splunk + Jira/ServiceNow + RAG) → Azure OpenAI syn
Predictive monitoring	Background loop polls Splunk trend data every N seconds → 3 statistical thresholds detect anomalies –
RAG knowledge base	Runbook documents pre-embedded into FAISS at startup → semantically searched at triage time → top
MCP Gateway	Single tool-call bus between agents and all external systems — 9 registered tools, all swappable for rea